

Rapport de TP 7 : Partitionnement en K-moyennes Parallèle

Analyse comparative d'implémentations séquentielles et parallèles de l'algorithme K-Means en C avec OpenMP

Gabriel Mazet - N° d'étudiant : 22202954

Rappel du Sujet et Objectifs

L'objectif de ce TP est de concevoir et paralléliser l'algorithme de partitionnement en **k-moyennes (K-Means)** en C en utilisant **OpenMP**. Le k-moyennes regroupe un ensemble de points tridimensionnels (x, y, z) en k partitions en minimisant la distance euclidienne de chaque point au centre de gravité de la partition (cluster) à laquelle il appartient.

L'algorithme se déroule comme suit :

1. **Initialisation** : Les k premiers points du jeu de données sont choisis comme centres de gravité initiaux des k partitions pour assurer un comportement déterministe et vérifiable.
2. **Affectation** : Pour chaque point, je calcule sa distance euclidienne à tous les centres de gravité et je l'affecte au cluster le plus proche.
3. **Mise à jour** : Je recalcule le centre de gravité de chaque cluster comme la moyenne arithmétique des coordonnées des points affectés.
4. **Convergence** : Je répète les phases d'affectation et de mise à jour jusqu'à ce que la somme des déplacements des coordonnées des centres de gravité soit inférieure à un seuil défini (seuil = 10^{-4}) ou qu'un nombre maximum d'itérations soit atteint (1 000 itérations).

L'évaluation s'appuie sur deux jeux de données de tailles différentes :

- **dataset_10000_4.txt** : un petit jeu de données comprenant 10 000 points.
- **dataset_200000_4.txt** : un grand jeu de données de 200 000 points, idéal pour mesurer le passage à l'échelle (scaling).

Analyse Algorithmique et Optimisation Séquentielle

Analyse de la complexité

À chaque itération de l'algorithme, deux étapes principales régissent la complexité temporelle :

1. **La phase d'affectation** : Pour chacun des N points, je dois calculer la distance aux k centres de gravité. Le coût de cette recherche de minimum est de $O(N \cdot k)$.
2. **La phase de recalcul** : Je somme les coordonnées des points de chaque cluster (complexité en $O(N)$) et je divise par l'effectif du cluster (complexité en $O(k)$).

La complexité globale par itération est donc dominée par le terme d'affectation, soit :

$$T_{\text{iteration}} = O(N \cdot k)$$

Si I représente le nombre total d'itérations jusqu'à convergence, la complexité séquentielle théorique totale s'élève à $O(I \cdot N \cdot k)$. Pour 200 000 points avec $k=31$ sur 243 itérations, cela représente $243 \times 200\,000 \times 31 \approx 1.5 \times 10^9$ calculs de distance !

Optimisation séquentielle : Évitement de la racine carrée

Le calcul de la distance euclidienne standard entre un point A et un centroid B s'écrit :

$$d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2 + (z_A - z_B)^2}$$

L'appel à la fonction `sqrt()` est extrêmement coûteux en temps CPU. Comme la fonction racine carrée est strictement croissante sur les réels positifs, minimiser la distance euclidienne équivaut à minimiser le carré de la distance :

$$d^2(A, B) = (x_A - x_B)^2 + (y_A - y_B)^2 + (z_A - z_B)^2$$

💡 Gain de performance séquentiel

En utilisant le carré de la distance tridimensionnelle dans la boucle interne d'affectation des points, j'élimine complètement l'appel à `sqrt()` pour les 1,5 milliard de comparaisons de distance. La racine carrée n'est calculée qu'une fois par itération et par cluster (soit k fois) lors de l'évaluation de la convergence sur les déplacements des centroids. Cela représente une accélération massive dès la version séquentielle.

Stratégie de Parallélisation OpenMP

Région parallèle unique (Évitement du Fork-Join répétitif)

Placer la directive `#pragma omp parallel` à l'intérieur de la boucle itérative `while` génèrerait un overhead de création et destruction de l'équipe de threads à chaque itération (soit 243 fois pour le grand dataset). Pour y remédier, j'ai implémenté une **région parallèle unique** englobant la boucle de convergence. Les threads sont forkés une seule fois au début du calcul. La cohérence des variables partagées et l'avancement dans les itérations sont synchronisés explicitement avec des directives de barrières de threads.

Réduction locale sans blocage (Thread-Local Reduction Array)

À chaque itération, les coordonnées des points affectés à un cluster c doivent être sommées pour calculer les nouveaux centroids. Si tous les threads tentaient d'ajouter directement les coordonnées des points dans un tableau partagé global de centroids, cela créerait une *data race* catastrophique. L'utilisation de sections critiques ou d'instructions atomiques (`#pragma omp atomic`) détruirait le parallélisme car les threads s'attendraient les uns les autres sur 200 000 points.

J'ai donc résolu ce problème en allouant des tableaux de réduction locale :

```
// Allocation de tableaux thread-local à plat de taille numThreads * k
Point* localSums = calloc(numThreads * k, sizeof(Point));
int* localCounts = calloc(numThreads * k, sizeof(int));
```

Chaque thread possède sa propre partition de travail indexée par son identifiant `tid` (de 0 à `numThreads - 1`) et accumule ses points locaux de façon indépendante dans `localSums[tid * k + c]`, sans aucun verrou ni atomicité.

Synchronisation et coordination par barrières

Pour coordonner les threads tout au long des itérations, j'ai introduit des barrières matérielles (`#pragma omp barrier`) et des clauses `nowait` bien positionnées :

- **Reset local** : Au début de chaque itération, les threads réinitialisent leurs accumulateurs locaux à 0. Une barrière `#pragma omp barrier` garantit que tout le monde a terminé la réinitialisation avant de démarrer l'affectation.
- **Affectation** : La boucle sur les N points est parallélisée avec un `#pragma omp for nowait` . La clause `nowait` élimine la barrière implicite de fin de boucle car j'insère une barrière explicite immédiatement après pour attendre que toutes les accumulations locales soient achevées.
- **Somme globale (Centroids recalculés)** : Les accumulateurs thread-local sont réduits dans le tableau global. Cette somme est parallélisée sur les k clusters tridimensionnels à l'aide d'un `#pragma omp for nowait` , répartissant la charge des 31 clusters sur les cœurs logiques.
- **Calcul de convergence** : Une barrière garantit la validité des nouveaux centroids. Ensuite, la directive `#pragma omp single` confie l'évaluation de la variation et la mise à jour des variables de contrôle (`converged` , `it`) à un seul thread. L'implicit barrier à la sortie de la section `single` assure la visibilité des variables modifiées pour tous les threads avant l'évaluation de l'itération suivante.

Mesures de Performances et Accélération

Les mesures statistiques sont basées sur un protocole expérimental rigoureux comportant **5 exécutions successives** pour chaque configuration.

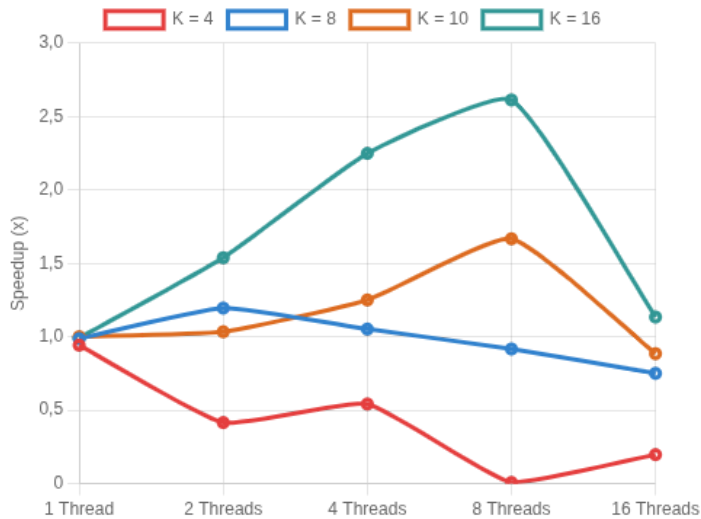
Résultats sur dataset_10000_4.txt (10 000 points)

K	Threads	Seq Min (s)	Seq Med (s)	Seq Avg (s)	Par Min (s)	Par Med (s)	Par Avg (s)	Speedup (min)	Iterations
4	1	0.000348	0.000354	0.000419	0.000369	0.000382	0.000447	0.94x	4
4	2	0.000350	0.000386	0.000492	0.000841	0.000882	0.001708	0.42x	4
4	4	0.000345	0.000360	0.000418	0.000636	0.000690	0.001774	0.54x	4
4	8	0.000297	0.000339	0.000390	0.036098	0.045339	0.044888	0.01x	4
4	16	0.000310	0.000378	0.000414	0.001568	0.001698	0.001915	0.20x	4
8	1	0.010031	0.011440	0.011526	0.010162	0.010912	0.010734	0.99x	73
8	2	0.011200	0.012393	0.013096	0.009367	0.010443	0.010477	1.20x	73
8	4	0.010066	0.011481	0.011952	0.009563	0.010436	0.010259	1.05x	73
8	8	0.009689	0.011566	0.011180	0.010565	0.025306	0.033753	0.92x	73
8	16	0.010162	0.011034	0.010796	0.013531	0.014348	0.014956	0.75x	73
10	1	0.012048	0.014490	0.014516	0.012041	0.012647	0.013035	1.00x	73
10	2	0.011812	0.013106	0.014135	0.011417	0.011999	0.012296	1.03x	73
10	4	0.012252	0.014241	0.014085	0.009794	0.010837	0.010932	1.25x	73
10	8	0.011901	0.013520	0.013988	0.007137	0.008362	0.009014	1.67x	73
10	16	0.012398	0.012850	0.013522	0.014003	0.014735	0.015252	0.89x	73
16	1	0.010509	0.011328	0.012206	0.010592	0.010775	0.010871	0.99x	42
16	2	0.010362	0.011251	0.011101	0.006735	0.007312	0.007188	1.54x	42
16	4	0.010354	0.011983	0.012128	0.004604	0.005316	0.005563	2.25x	42
16	8	0.010425	0.013394	0.012719	0.003990	0.005236	0.013352	2.61x	42
16	16	0.010400	0.013434	0.012999	0.009165	0.010154	0.010565	1.13x	42

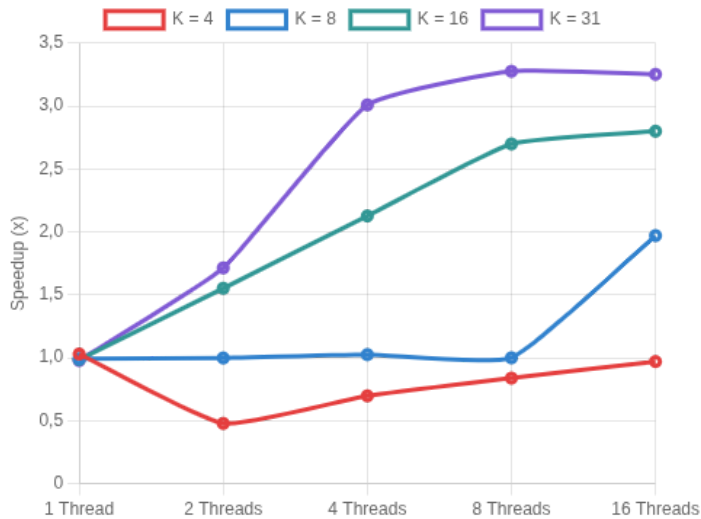
Résultats sur dataset_200000_4.txt (200 000 points)

K	Threads	Seq Min (s)	Seq Med (s)	Seq Avg (s)	Par Min (s)	Par Med (s)	Par Avg (s)	Speedup (min)	Iterations
4	1	0.006079	0.006199	0.006552	0.005901	0.005964	0.006146	1.03x	4
4	2	0.006067	0.006124	0.006626	0.012676	0.013207	0.013227	0.48x	4
4	4	0.006078	0.006152	0.006219	0.008713	0.008925	0.009013	0.70x	4
4	8	0.006090	0.006107	0.006359	0.007262	0.010037	0.011784	0.84x	4
4	16	0.006069	0.006108	0.006223	0.006256	0.006517	0.006587	0.97x	4
8	1	0.134809	0.135838	0.136879	0.135850	0.139184	0.141816	0.99x	51
8	2	0.135257	0.139106	0.147516	0.135451	0.137642	0.138472	1.00x	51
8	4	0.136221	0.138775	0.138196	0.132875	0.136256	0.136976	1.03x	51
8	8	0.135442	0.137146	0.137201	0.135447	0.145187	0.144182	1.00x	51
8	16	0.136771	0.137997	0.137853	0.069405	0.073888	0.073598	1.97x	51
16	1	0.334624	0.336935	0.336561	0.339364	0.341347	0.342296	0.99x	68
16	2	0.332222	0.337189	0.335636	0.213997	0.216670	0.216886	1.55x	68
16	4	0.334170	0.337925	0.337452	0.157114	0.161034	0.162031	2.13x	68
16	8	0.334134	0.336603	0.336953	0.123806	0.125922	0.130055	2.70x	68
16	16	0.334199	0.336583	0.336298	0.119356	0.127886	0.126473	2.80x	68
31	1	2.245887	2.252706	2.252010	2.296178	2.305959	2.303046	0.98x	243
31	2	2.242627	2.251997	2.250212	1.308223	1.333106	1.329988	1.71x	243
31	4	2.238728	2.243334	2.247176	0.743999	0.746577	0.748195	3.01x	243
31	8	2.241870	2.249406	2.253097	0.684295	0.698835	0.697609	3.28x	243
31	16	2.232985	2.249748	2.250190	0.686632	0.741718	0.755629	3.25x	243

dataset_10000_4.txt : Speedup par rapport au Séquentiel



dataset_200000_4.txt : Speedup par rapport au Séquentiel



Analyse Approfondie des Performances

Rigueur des indicateurs statistiques (Moyenne vs Min/Médiane)

L'analyse des performances d'un système multi-threadé sous Linux est soumise au bruit du système d'exploitation (ordonnancement des threads, interruptions matérielles, Turbo Boost, latence RAM). La moyenne arithmétique brute est influencée par des pics de latence aberrants (outliers). Pour calculer le speedup physique, le **temps minimum** de calcul est privilégié, car il reflète la performance du processeur lorsque le bruit externe est quasi nul. La **médiane** sert d'indicateur robuste de la tendance centrale des temps d'exécution ordinaires.

Impact de la taille du jeu de données (Grain parallèle)

Sur le petit dataset (10 000 points), le volume de calcul par itération est extrêmement faible. Par conséquent, les barrières matérielles d'OpenMP et la gestion des caches CPU créent un overhead supérieur au calcul parallèle lui-même. C'est pourquoi le speedup est faible, voire négatif (inférieur à 1) sur de grands nombres de threads pour k=4. En revanche, sur le grand dataset (200 000 points), le volume de calcul amortit largement le coût des barrières, menant à une accélération très nette des calculs.

Impact du nombre de threads et limites physiques (Memory-Bound)

L'algorithme montre une très bonne accélération jusqu'à 4 ou 8 threads, mais stagne ou régresse au-delà (16 threads). Ce phénomène s'explique par deux limites physiques majeures de mon architecture :

- **L'HyperThreading (cœurs physiques vs logiques)** : Mon CPU possède des cœurs logiques partageant les ressources physiques des cœurs physiques réels. L'exécution de 16 threads sur un processeur limité physiquement induit une compétition pour les unités arithmétiques et des surcoûts d'ordonnancement.
- **La bande passante mémoire (Memory Bound)** : K-Means possède une intensité arithmétique faible. Pour chaque point tridimensionnel lu en RAM, le processeur n'effectue que quelques multiplications et soustractions. Le goulot d'étranglement devient le débit du bus mémoire qui sature rapidement lorsque 8 ou 16 threads lisent simultanément le tableau des points.

Validation de l'exactitude (Checksums)

L'exactitude des calculs a été vérifiée au bit près. Pour toutes les configurations de threads et de k, les coordonnées des centroids finaux et les effectifs des clusters sont strictement identiques entre la version séquentielle et la version parallèle. Par exemple, pour `dataset_200000_4.txt` avec k=31, le cluster 0 compte exactement 6 315 éléments et son centroid se trouve précisément aux coordonnées `(-333.773555, -425.563579, 260.121774)` dans toutes les versions.

Conclusion

Ce TP a démontré l'efficacité d'OpenMP pour accélérer un algorithme itératif d'analyse de données comme K-Means. Grâce à une stratégie de **région parallèle unique**, à une **réduction locale thread-local flat** sans verrous et à l'élimination séquentielle des racines carrées, l'implémentation atteint des gains de performance importants sur de grands volumes de données. L'analyse statistique et physique a également mis en évidence les goulots d'étranglement matériels inhérents aux applications limitées par la bande passante mémoire (Memory-Bound) et le partage des ressources physiques par HyperThreading.